LLM ARCHITECTURE & TRAINING

Topics on NLP & LLM
- ADDI Laboratory -

Rarasmaya Indraswari
(WA) 081331818454
raras@its.ac.id

Contents

- LLM Architecture
- Training Pipeline
- Knowledge Distillation

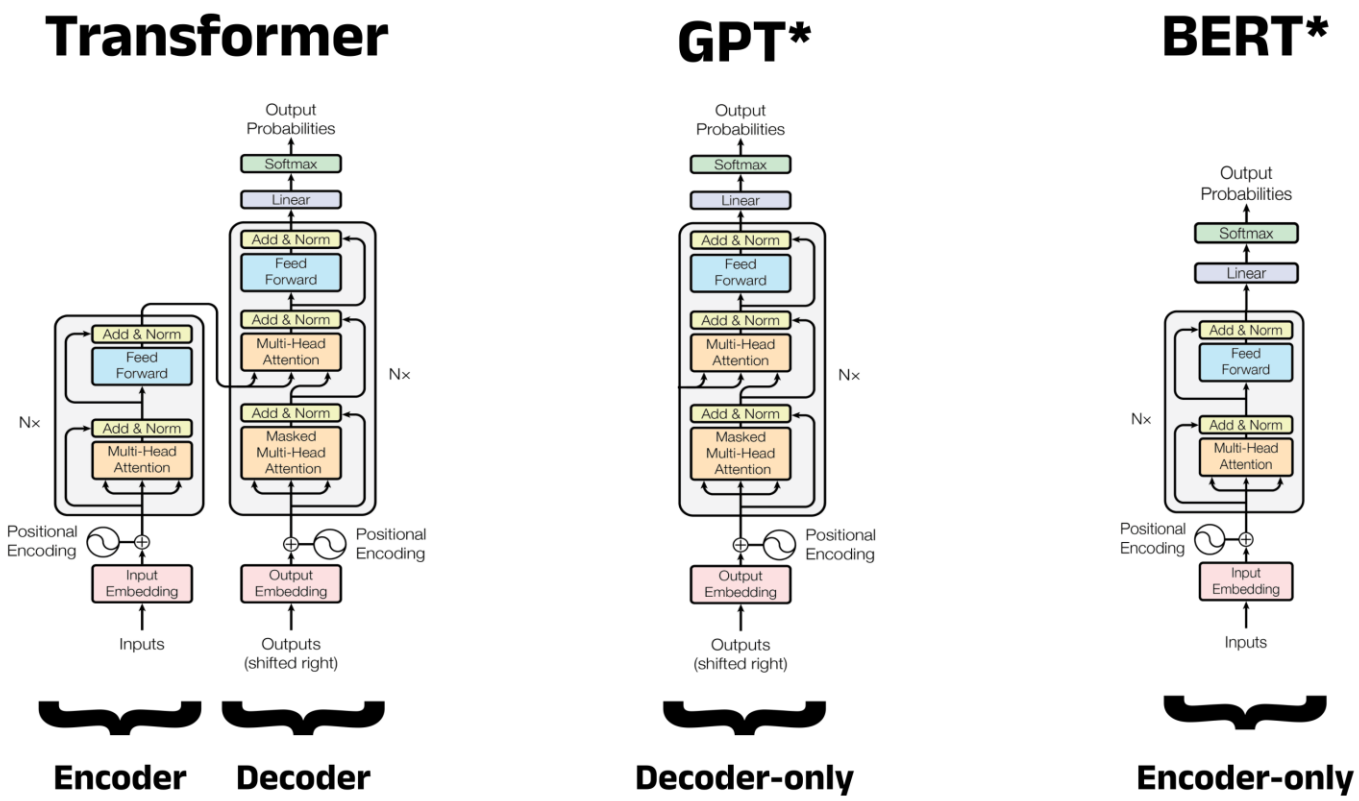
LLM Architecture

Overview

- Transformer: embedding → positional encoding → self-attention → feed-forward

Type	Model	Architecture Type	Task
Encoder-only	BERT, RoBERTa, DistilBERT	Understanding	Klasifikasi, QA, embedding
Decoder-only	GPT-2, GPT-Neo, GPT-J, LLaMA	Generation	Chat, completion, story
Encoder-Decoder	T5, BART, FLAN-T5	Seq2Seq	Summarization, translation, QA generatif

Architecture



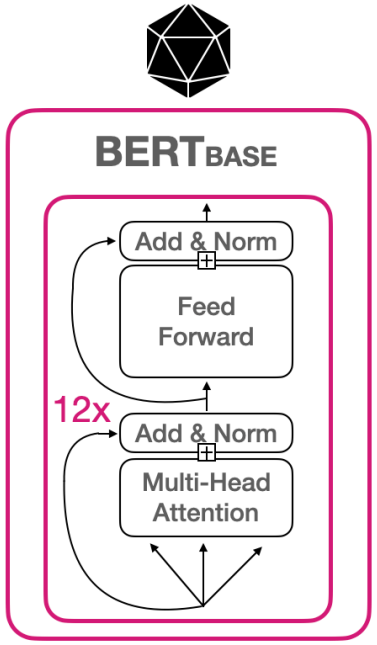
*Illustrative example, exact model architecture may vary slightly

BERT

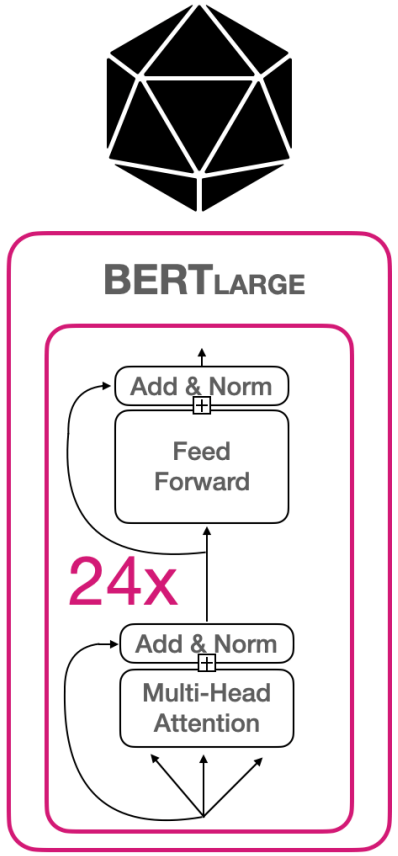
- BERT = Bidirectional Encoder Representations from Transformers
- Task:
  - Sentiment Analysis
  - Question answering
  - Text prediction
  - Text generation
  - Summarization
  - Polysemy resolution: Can differentiate words that have multiple meanings (like ‘bank’) based on the surrounding text
- Using MLM (Masked Language Model) that enables/enforces bidirectional learning from text by masking (hiding) a word in a sentence and forcing BERT to bidirectionally use the words on either side of the covered word to predict the masked word.

BERT Architecture

BERT Size & Architecture



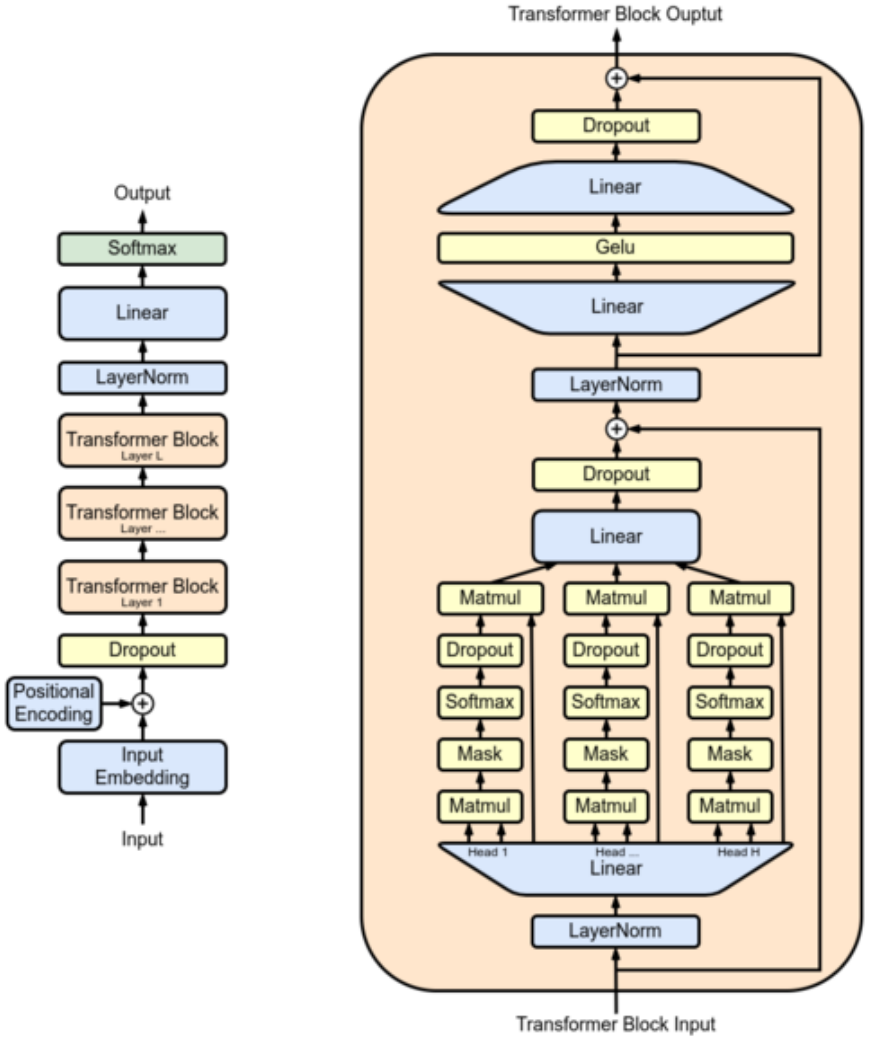
110M Parameters



340M Parameters

https://huggingface.co/blog/bert-101

GPT Architecture

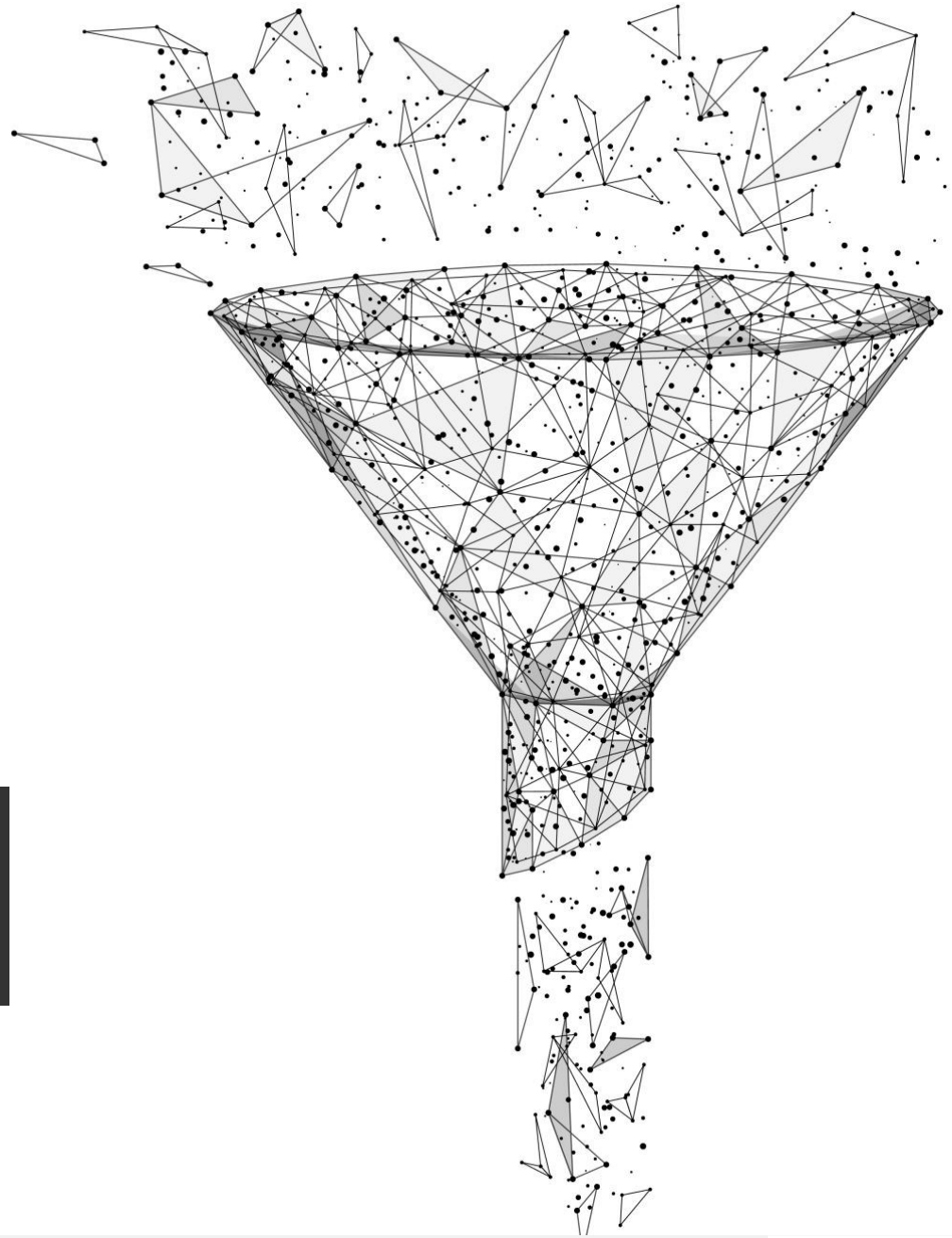


GeLU: Gaussian Error Linear Unit

- To solve dying ReLU problem: neuron can stuck with 0 value
- Usually used in Transformer-based architecture

File:Full GPT architecture.png - Wikimedia Commons

Training Pipeline



1. Pre-training

- Objective: language understanding
- Massive dataset: Common Crawl, Wikipedia, BooksCorpus
- Pretrained LLM = general knowledge

2. Fine-tuning

- Menyesuaikan model untuk domain tertentu: medis, hukum, dsb.
- Full Fine-Tuning:
  - Semua parameter di-update
  - Butuh GPU besar dan dataset besar sehingga biasanya hanya dilakukan oleh big tech labs
- Masalah pada full fine-tuning: Model besar (miliaran parameter) → tidak efisien untuk diupdate penuh
- Parameter-Efficient Tuning Techniques (PEFT):
  - **Adapter tuning:** menambahkan trainable layer kecil baru di setiap blok Transformer
  - **Prefix-tuning:** menambahkan vektor continuous (prefix) yang disisipkan ke setiap layer attention
  - **Prompt-tuning / soft prompting:** mempelajari vektor prompt yang hanya ditempatkan di input embedding sebagai token virtual awal (learned prompt)
  - **LoRA (Low-Rank Adaptation):** menyisipkan trainable matriks adaptor kecil di dalam layer attention → $W' = W + AB$ di mana bobot besar W difreeze dan hanya matriks kecil A dan B yang dilatih

PEFT

Aspek	Full Fine-Tuning	Adapter Tuning	Prefix Tuning	Prompt Tuning	LoRA (Low-Rank Adaptation)
Konsep Dasar	Melatih ulang semua parameter model	Menambahkan modul kecil (adapter layers) di antara lapisan transformer	Menambahkan vektor awalan (prefix vectors) ke tiap layer attention	Menambahkan vektor prompt khusus di input embedding	Mengganti sebagian weight matrices dengan dekomposisi low-rank
Parameter yang Diubah	100% dari parameter model	Hanya parameter adapter (biasanya <5%)	Hanya parameter prefix (biasanya <1%)	Hanya prompt embedding (biasanya <0.1%)	Hanya matriks low-rank tambahan (biasanya <1–2%)
Perubahan Arsitektur Model	Tidak ada (hanya bobot diperbarui)	Tambah modul adapter per layer	Tidak ubah arsitektur, hanya modifikasi input layer dan attention	Tidak ubah arsitektur, hanya tambahkan embedding awal	Tidak ubah arsitektur utama, hanya tambahkan matriks kecil pada layer linear
Kebutuhan Komputasi & Memori	Sangat tinggi (butuh GPU besar)	Menengah (hemat 90–95%)	Rendah	Sangat rendah	Rendah–menengah
Kelebihan	Akurasi optimal; fleksibel untuk semua domain	Modular; bisa digunakan untuk banyak tugas tanpa retraining penuh	Hemat parameter; efektif untuk generasi teks panjang	Paling ringan; cocok untuk banyak tugas kecil	Efisien, hasil hampir sebaik full fine-tuning; mudah diintegrasikan
Kekurangan	Biaya besar, sulit disimpan & dikelola	Perlu modifikasi arsitektur; bisa menambah latensi	Kurang stabil untuk tugas non-generatif	Kinerja menurun jika data kompleks	Perlu tuning rank dan skala; tidak selalu stabil di semua layer
Contoh Implementasi	Fine-tuning BERT untuk klasifikasi teks	Adapter-BERT, AdapterHub	Prefix-Tuning GPT-2	Prompt Tuning T5	LoRA-LLAMA, LoRA-GPT
Kapan Digunakan	Jika dataset besar & infrastruktur kuat	Jika ingin multi-task model efisien	Jika tugas generatif (dialog, story)	Jika sumber daya terbatas	Jika butuh efisiensi tinggi tanpa banyak kehilangan performa

3. Alignment

- Process to make the model not only knowledgeable but also helpful, honest, and safe.
- Main processes:
  - Instruction Tuning
    - The model is trained on datasets of (instruction, response) pairs, such as: “Explain quantum computing in simple terms.” → Model generates clear, short explanation.
    - This helps the model follow human commands naturally.
  - Reinforcement Learning from Human Feedback (RLHF)
    - Multiple model outputs are ranked by humans (or preference models), and the model is optimized to produce the most preferred response.
    - This is how models like ChatGPT became conversationally aligned.
- Example: GPT-3.5 → GPT-4
- Reinforcement Learning from Human Feedback (RLHF) Explained

Knowledge Distillation

Problems of Big Model

- Problems of big model:
  - Konsumsi energi tinggi
  - Latensi inferensi lambat
  - Sulit di-deploy untuk instansi Pendidikan maupun startup
- Solution: model compression
  - Pruning: menghapus bobot tak penting
  - Quantization: mengurangi presisi (float32 → int8)
  - Knowledge Distillation: melatih model kecil (student) agar meniru model besar (teacher)
- Contoh model hasil distillation: DistilBERT, TinyBERT, DistilGPT2

Knowledge Distillation

- Knowledge distillation adalah teknik di bidang machine learning/deep learning di mana sebuah model besar (teacher) yang sudah dilatih digunakan untuk membantu melatih sebuah model kecil (student) agar memiliki performa mendekati teacher, namun dengan ukuran dan kompleksitas lebih ringan.
- Teacher menghasilkan soft targets (distribusi probabilitas output) atau representasi internal yang kemudian diimitasi oleh student.
- Tujuan: mengurangi ukuran model, mempercepat inferensi, mengurangi kebutuhan komputasi sambil mempertahankan akurasi.
- Teknik ini banyak diterapkan di NLP (contoh: DistilBERT) dan computer vision.

Mekanisme

- Teacher Model: Model besar yang sudah dilatih, menghasilkan output $pteacher$ (distribusi probabilitas atau nilai logit) untuk input tertentu.
- Student Model: Model lebih kecil yang akan dilatih untuk meniru output teacher.
- Loss Function: Kombinasi antara loss terhadap label asli (hard targets) dan loss terhadap output teacher (soft targets); misalnya:

$$L = \alpha \cdot \mathcal{L}_{\text{CE}}(y_{\text{true}}, p_{\text{student}}) + (1 - \alpha) \cdot T^2 \cdot \mathcal{L}_{\text{KL}}(p_{\text{teacher}}(T), p_{\text{student}}(T))$$

  di mana T adalah *temperature* untuk memperhalus distribusi output teacher.
- Student dilatih hingga outputnya mendekati teacher → ukuran lebih kecil, inference lebih cepat.

Referensi

- Paper: https://arxiv.org/abs/1503.02531
- Video: (9) Knowledge Distillation: How LLMs train each other - YouTube

Thank You